UNIT III
Designing User Interfaces with Layouts

Building Layouts using Jetpack Compose

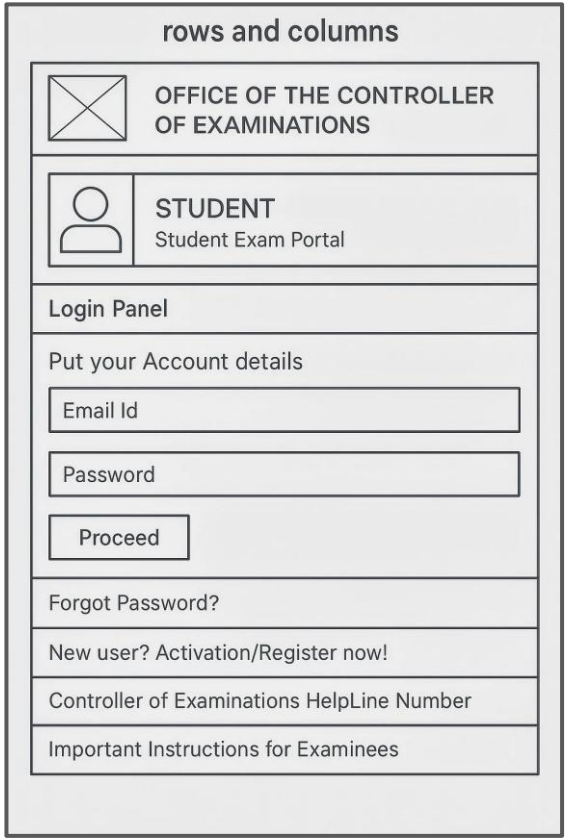
Designing User Interfaces with Layouts

A layout refers to the structure that defines the user interface (UI) of an app screen.

It is similar to designing the architecture of the house before starting the actual construction work.

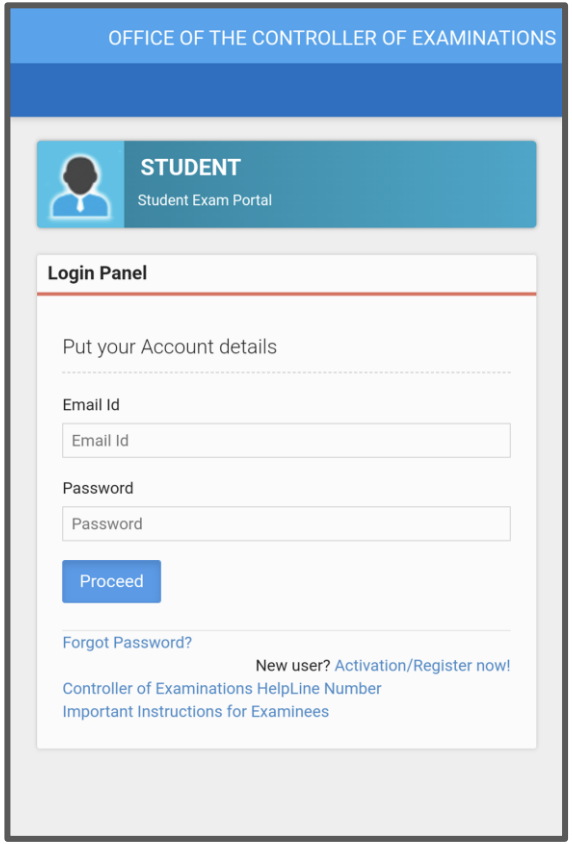
Designing User Interfaces with Layouts

Layout



Designing User Interfaces with Layouts

Activity



Designing User Interfaces with Layouts

Package: androidx.compose.foundation.layout.*

Designing User Interfaces with Layouts

Without Layout

Code

```
@Composable
fun ArtistCard() {
    Text("Alfred Sisley")
    Text("3 minutes ago")
}
```

Output

Alfred Sisley
3 minutes ago

Designing User Interfaces with Layouts

With Layout

Code

```
@Composable
fun ArtistCardColumn() {
    Column {
        Text("Alfred Sisley")
        Text("3 minutes ago")
    }
}
```

Output

Alfred Sisley

3 minutes ago

Designing User Interfaces with Layouts

With Layout

Code 1

```
@Composable
fun ArtistCardColumn() {
    Column {
        Text("Alfred Sisley")
        Text("3 minutes ago")
    }
}
```

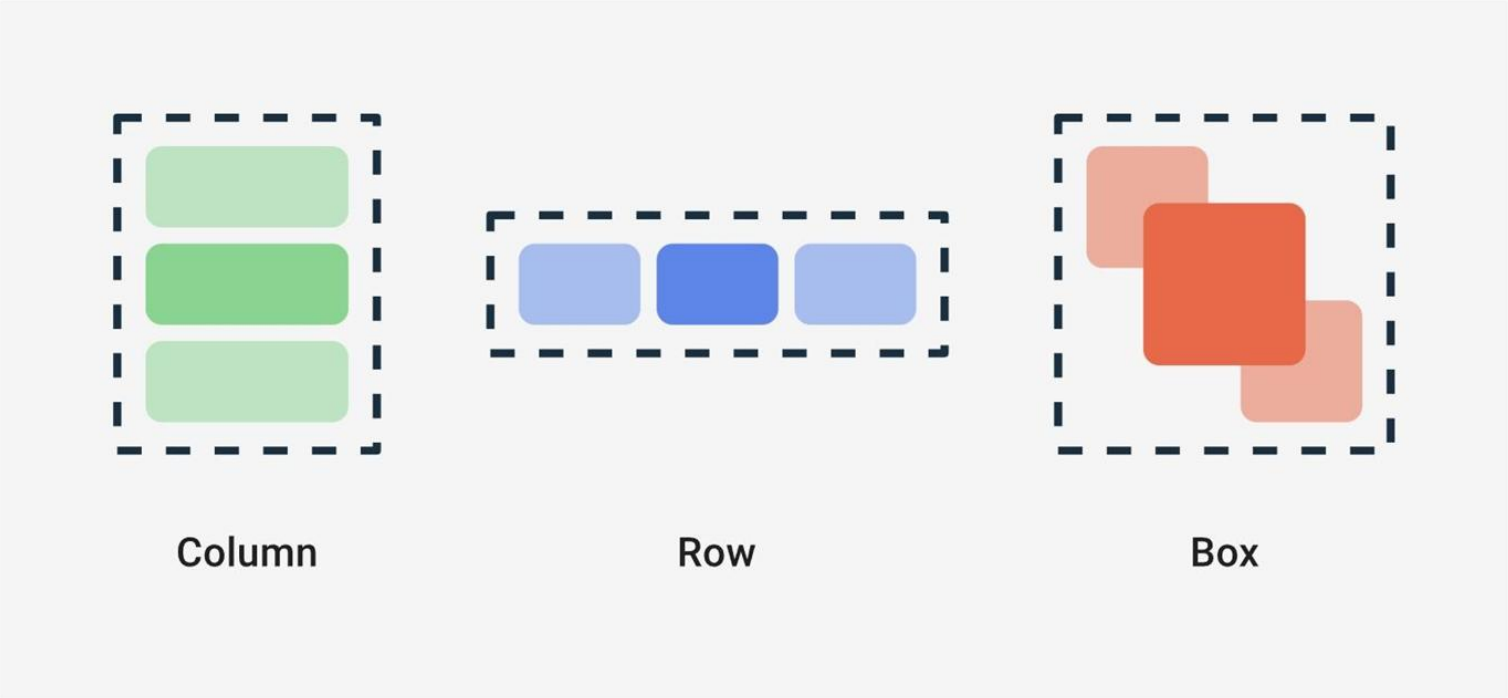
Without Layout

Code 2

```
@Composable
fun ArtistCard() {
    Text("Alfred Sisley")
    Text("3 minutes ago")
}
```

Designing User Interfaces with Layouts

Often these building blocks are all you need. You can write your own composable function to combine these layouts into a more elaborate layout that suits your app.



Designing User Interfaces with Layouts

Modifier

Before proceeding further, we need to discuss one more important term, **Modifier.**

→

A **Modifier** in Jetpack Compose is **an object** that extends or changes the behavior of a composable without changing its core functionality.

It can control size, layout, appearance, interaction, and more.

Designing User Interfaces with Layouts

Modifier

```
Text(
    text = "Hello, Modifier!",
    modifier = Modifier
        .padding(16.dp)
        .background(Color.LightGray)
        .fillMaxWidth()
)
```

Designing User Interfaces with Layouts

Modifier

padding() – Add space around a composable.

margin() – (Not built-in; typically done with padding or spacing composables)

size(), **width()**, **height()** – Set fixed size.

fillMaxWidth(), **fillMaxHeight()**, **fillMaxSize()** – Occupy available space.

wrapContentWidth(), **wrapContentHeight()** – Shrink to fit content.

Designing User Interfaces with Layouts

Modifier

background() – Add background color or rounded background.

border() – Add borders around the column.

align() – Align children within the column (used on individual child composables).

horizontalAlignment – Align all children in the column (parameter of Column()).

verticalArrangement – Control spacing between items (parameter of Column()).

Designing User Interfaces with Layouts

Column Layout:

The Column() composable in Jetpack Compose is used to arrange child composables vertically.

Designing User Interfaces with Layouts

Column Layout:

```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
        .background(Color.LightGray, shape = RoundedCornerShape(8.dp))
        .border(2.dp, Color.Gray, RoundedCornerShape(8.dp)),
    verticalArrangement = Arrangement.SpaceEvenly,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Name: Alice", modifier = Modifier.padding(8.dp))
    Text("Name: Bob", modifier = Modifier.padding(8.dp))
}
```

Designing User Interfaces with Layouts

Column Layout:

```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
        .background(Color.LightGray, shape = RoundedCornerShape(8.dp))
        .border(2.dp, Color.Gray, RoundedCornerShape(8.dp)),
    verticalArrangement = Arrangement.SpaceEvenly,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Name: Alice", modifier = Modifier.padding(8.dp))
    Text("Name: Bob", modifier = Modifier.padding(8.dp))
}
```

Modifier

Designing User Interfaces with Layouts

Column Layout:

```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
        .background(Color.LightGray, shape = RoundedCornerShape(8.dp))
        .border(2.dp, Color.Gray, RoundedCornerShape(8.dp)),
    verticalArrangement = Arrangement.SpaceEvenly,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Name: Alice", modifier = Modifier.padding(8.dp))
    Text("Name: Bob", modifier = Modifier.padding(8.dp))
}
```

Column Parameter

Designing User Interfaces with Layouts

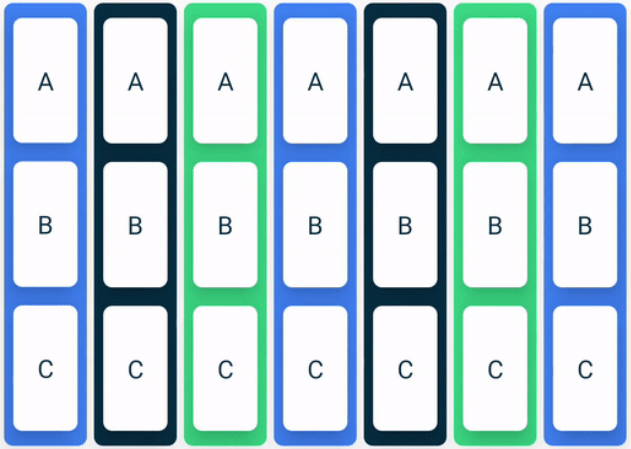
Column Layout:

```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
        .background(Color.LightGray, shape = RoundedCornerShape(8.dp))
        .border(2.dp, Color.Gray, RoundedCornerShape(8.dp)),
    verticalArrangement = Arrangement.SpaceEvenly,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Name: Alice", modifier = Modifier.padding(8.dp))
    Text("Name: Bob", modifier = Modifier.padding(8.dp))
}
```

Body

Column Layout:

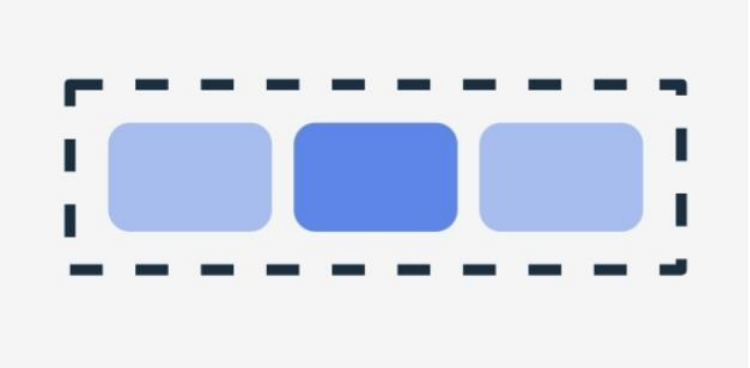
```
Column(
    modifier = Modifier
        .fillMaxWidth()
        .padding(16.dp)
        .background(Color.LightGray, shape = RoundedCornerShape(8.dp))
        .border(2.dp, Color.Gray, RoundedCornerShape(8.dp)),
    verticalArrangement = Arrangement.SpaceEvenly,
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Name: Alice", modifier = Modifier.padding(8.dp))
    Text("Name: Bob", modifier = Modifier.padding(8.dp))
}
```



Designing User Interfaces with Layouts

Row Layout:

Row is a composable that arranges its children horizontally in a single line, from left to right.



Designing User Interfaces with Layouts

Row Layout:

```
@Composable
fun Row(
    modifier: Modifier = Modifier,
    horizontalArrangement: Arrangement.Horizontal = Arrangement.Start,
    verticalAlignment: Alignment.Vertical = Alignment.Top,
    content: @Composable RowScope.() -> Unit
)
```

Designing User Interfaces with Layouts

Row Layout:

1. **horizontalArrangement:** Arrangement.Horizontal → Controls how the children are spaced horizontally within the Row. Common options are listed below.

Arrangement.Start (default) – places children at the start.

Arrangement.Center – centers children horizontally.

Arrangement.End – aligns children to the end.

Arrangement.SpaceBetween – equal space between items.

Designing User Interfaces with Layouts

Row Layout:

Arrangement.SpaceAround – equal space around items.

Arrangement.SpaceEvenly – equal space before, between, after.

Designing User Interfaces with Layouts

Row Layout:

2. verticalAlignment: Alignment.Vertical → Controls how children align vertically within the Row.

Commons Options:

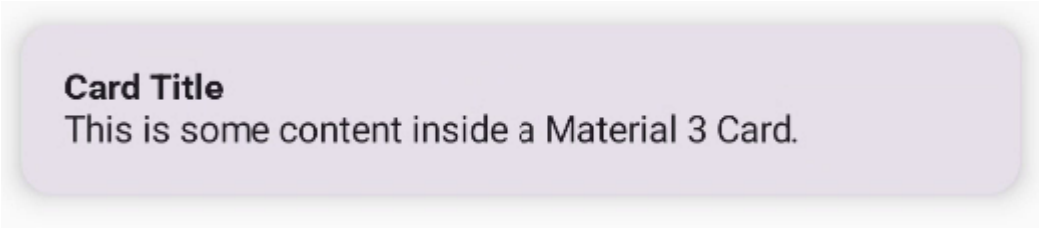
Alignment.Top – Aligns all children to the top of the tallest item (default)**.**

Alignment.CenterVertically – Aligns children vertically centered within the Row.

Alignment.Bottom – Aligns all children to the bottom of the tallest item**.**

Card() container

In Jetpack Compose, the **Card** is a specialized container that sits "on top" of other layouts. While you can build a card using just a Box or Column, the official Card component automatically handles Material Design standards like **elevation (shadows)** and **rounded corners**.



```
@Composable
fun SimpleCard() {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        shape = RoundedCornerShape(12.dp),
        elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text( text: "Card Title", fontWeight = FontWeight.Bold)
            Text( text: "This is some content inside a Material 3 Card.")
        }
    }
}
```

Card() container

In modern Android (2026), there are three specific card types:

Type	Function	Visual Style
Card	Filled Card	Subtly colored background, no heavy shadow.
ElevatedCard	Floating Card	Has a distinct shadow (best for high-priority items).
OutlinedCard	Bordered Card	No shadow, just a thin outline (clean and minimal).

Card() container

Parameters

Parameter	Type	Purpose
modifier	Modifier	Adjusts size, padding, and click behavior.
shape	Shape	Defines the corners (Rounded, Cut, or Circle).
colors	CardColors	Sets the background and content (text/icon) colors.
elevation	CardElevation	Controls the shadow intensity.
border	BorderStroke	Adds an outline (mostly used in OutlinedCard).
content	@Composable ColumnScope.() -> Unit	**Required.** What goes inside the card.

THANK YOU